

Solar Device Advance

An ESP32-based DIY solar/UPS controller designed to add solar management, power monitoring, automatic source switching, and configurable control functions to a Car Inverter Or conventional UPS.



⚠ Safety Warning: This project can involve battery currents, inverter output, and potentially lethal AC mains voltage. Do not work on energized circuits. Use appropriate fuses, isolation, protection, wiring, relay ratings, and enclosure clearances.

Video link :

Will be uploaded soon.

Project Support & Sponsors

The development of **Solar Device Advance** was made possible with support from **OSHWLab**, **EasyEDA**, and **JLCPCB**.

We sincerely thank these organizations for providing **complimentary PCBs and electronic components** used during the development and testing of this project.

[OSHWLab](#)

[EasyEDA](#)

[JLCPCB](#)

Thank you to **OSHWLab**, **EasyEDA**, and **JLCPCB** for supporting DIY and open-source hardware development.

Introduction

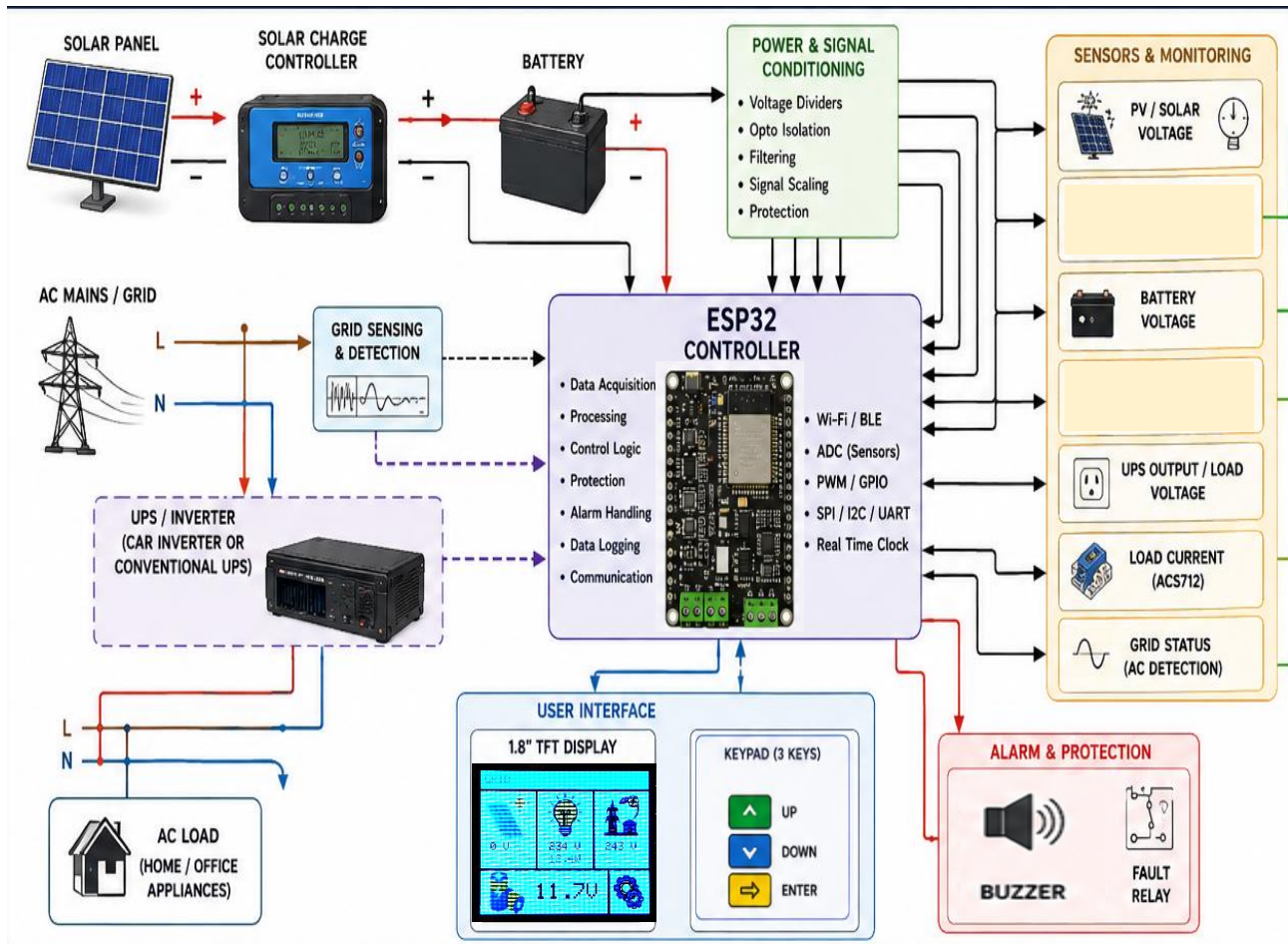
This Solar Device enables the user to built your own Solar System using a car inverter & charge controller OR to convert conventional UPS into Solar inverter.

Existing small/household UPS units provide battery-backed AC output but typically do not accept solar input or intelligently prioritise solar energy. Retrofitting these UPS units with a dedicated controller converts them to hybrid solar inverters at a fraction of the cost of buying a commercial solar inverter.

Parameter	Description
Controller	ESP32
Display	1.8" TFT
Current sensing	ACS712
Power sources	Solar / Grid / Battery
Inverter	Conventional UPS / Car inverter
Solar charging	External charge controller
Source switching	Relay-based
User interface	TFT + keypad
Connectivity	Wi-Fi / Bluetooth
Firmware	ESP32
Project type	DIY / Open Source

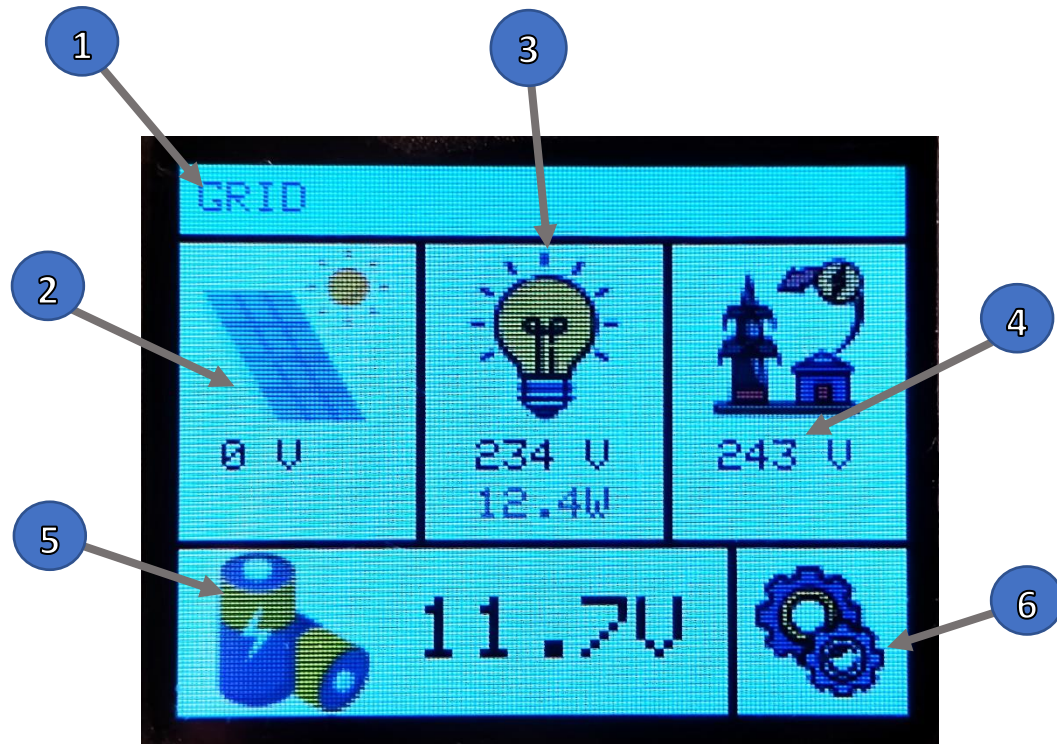
Project function

The controller integrates power monitoring, Solar charge regulation, automatic transfer switching (between grid, battery, and solar). It monitors PV voltage, battery voltage, Output/load current, and grid status; then makes intelligent switching.



User Interface

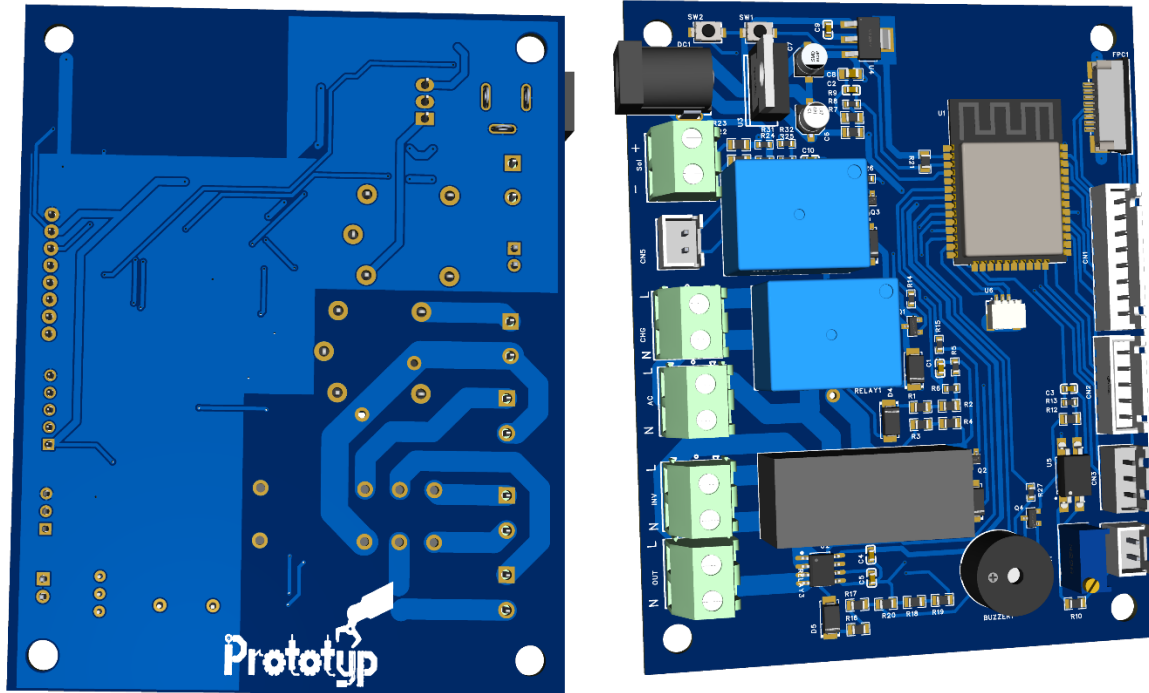
The Device provides a menu-based interface for system monitoring and configuration.



1. Working mode: Running on Grid or Battery
2. Solar Status
3. Load Status
4. Grid Status
5. Battery Status
6. Settings

Hardware description

1. Main Board



MicroController

The main controller is based on an **ESP32**.

The ESP32 handles:

- System-state monitoring
- Relay control
- Display control
- Keypad input
- Settings
- Fault handling
- Operating logic
- Communication functions

The ESP32 also provides Wi-Fi and Bluetooth/BLE hardware for possible future expansion and remote monitoring functions.

Power Monitoring

The controller monitors the electrical parameters.

These include the system's relevant:

- Solar/PV voltage
- Battery voltage
- AC/grid status
- UPS output/load condition
- Output Current measurements

The actual measurement circuits, scaling networks, sensor connections, and protection components are shown in the project schematic.

Current Measurement

Current measurement is performed using **ACS712 current sensors**.

The measured current is read by the ESP32 ADC and processed by the firmware.

Current measurements are used for system monitoring and control functions implemented in the System.

Sensor calibration is required to obtain accurate measurements.

Battery Monitoring

The controller monitors battery voltage.

Battery measurements can be used by the device to determine battery operating conditions and activate the configured battery protection behavior.

The actual battery limits and settings should be configured according to the battery type and the connected UPS/inverter system.

Grid Detection

The controller monitors the grid/AC input through the dedicated detection circuit.

The controller uses the grid status as one of the inputs for automatic operating-state decisions.

Solar Monitoring

Solar/PV conditions are monitored by the controller through the implemented voltage sensing circuits.

The controller uses these measurements to determine the available solar condition and display the relevant information to the user.

The controller can work with an external solar charge controller as part of the overall system.

Relay Control

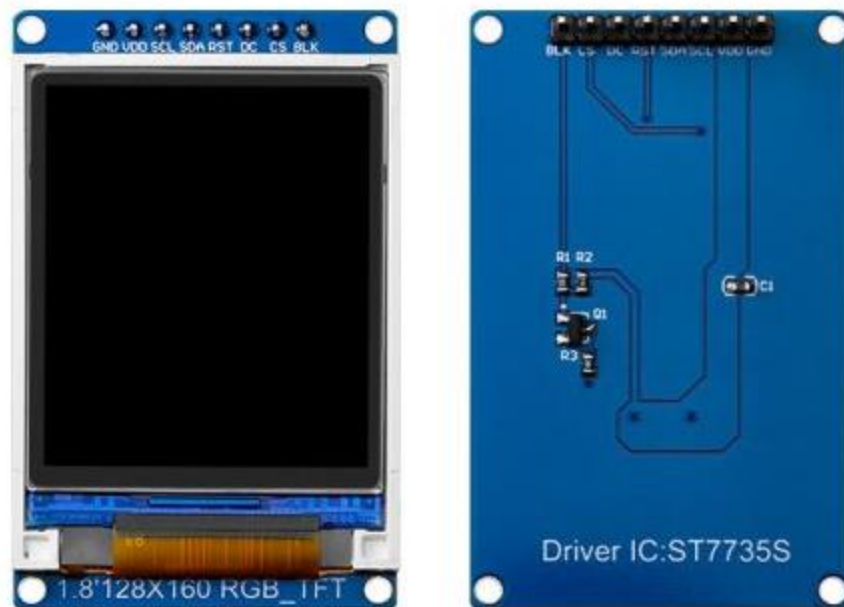
Relay outputs are controlled by the ESP32 through the relay-driver circuitry implemented on the PCB.

Relay 1 for AC charging
Relay 2 for Inverter control
Relay 3 for Load Shifting

The relay contacts must be rated appropriately for the connected voltage, current, load type, and inrush current.

Refer to the schematic for the exact circuit implementation.

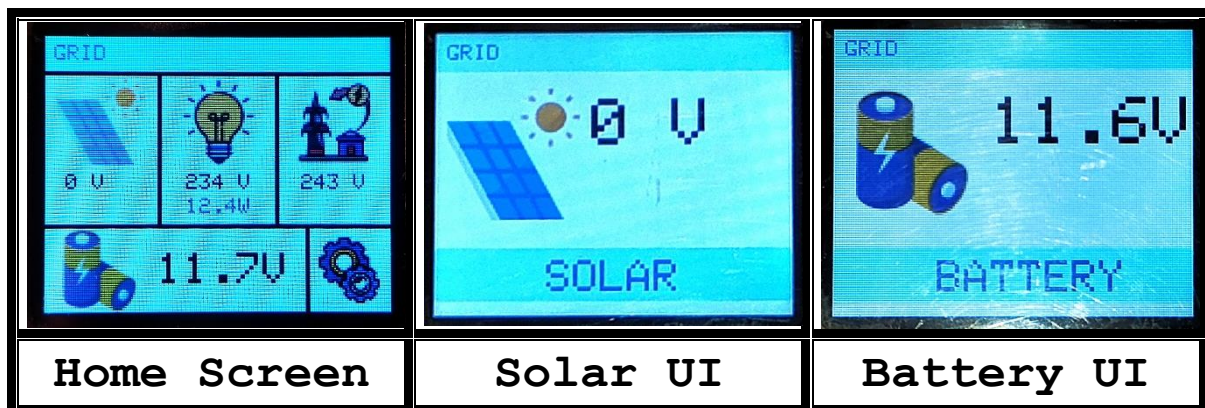
2. Display

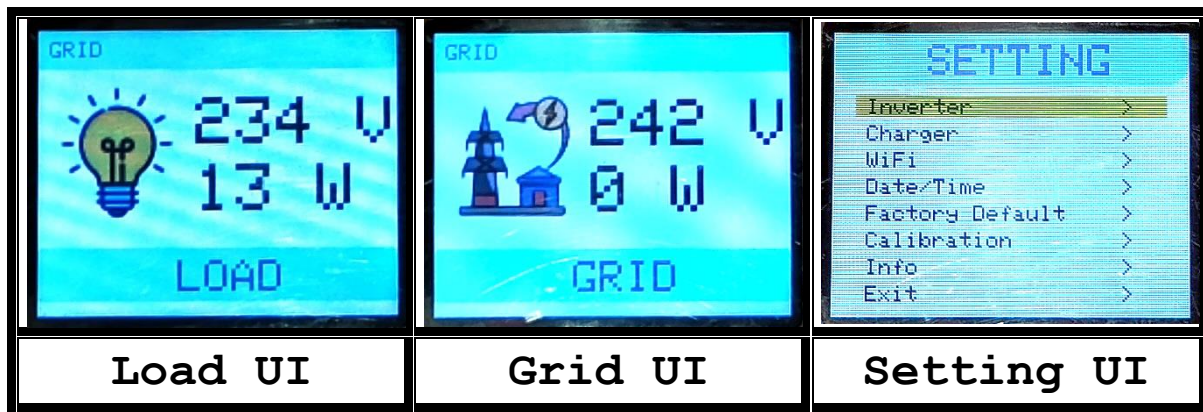


A **1.8-inch TFT display** provides the main user interface.

The display is used to show system information such as:

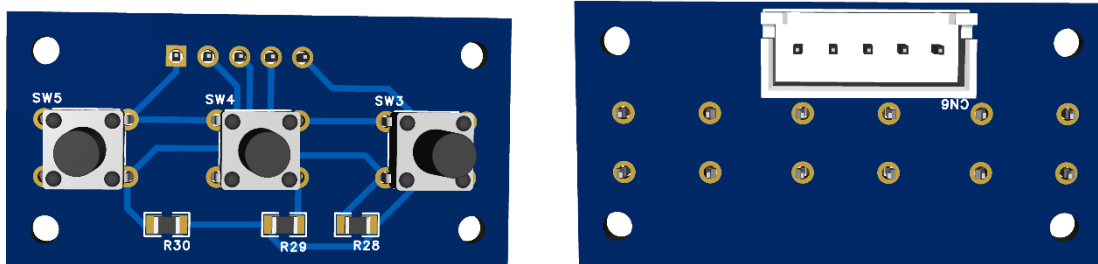
- Operating status
- Solar parameters
- Battery parameters
- Load/current information
- Grid status
- System state
- Settings





3. Keypad

A dedicated keypad is provided for navigation and configuration.



The keypad allows the user to:

- Navigate through menus
- Change settings
- Select operating options
- Confirm selections
- Return to previous menus
- Access monitoring pages

Key functions

Key	Function
Key 1	UP
Key 2	DOWN
Key 3	ENTER

Settings

The device provides configurable parameters through the keypad and TFT interface.

The available settings depend on the current firmware version.

Examples include:

- Operating mode
 - Battery thresholds
 - Solar-related thresholds
 - Switching parameters
 - Protection limits
 - Display settings
-

Firmware

The controller firmware is written on ArduinoIDE for the ESP32.

The latest firmware is available in the GitHub repository.

GitHub

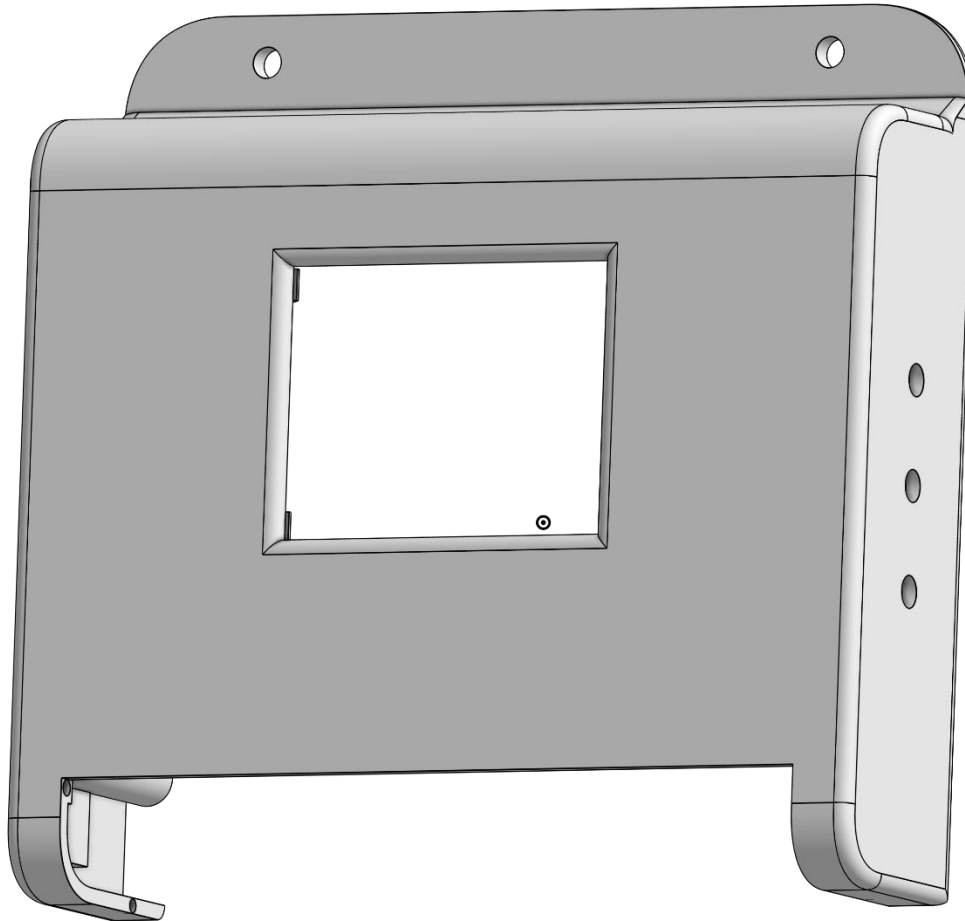
[Solar Device Advance – GitHub Repository](#)

Complete process to upload the firmware is described in the document:

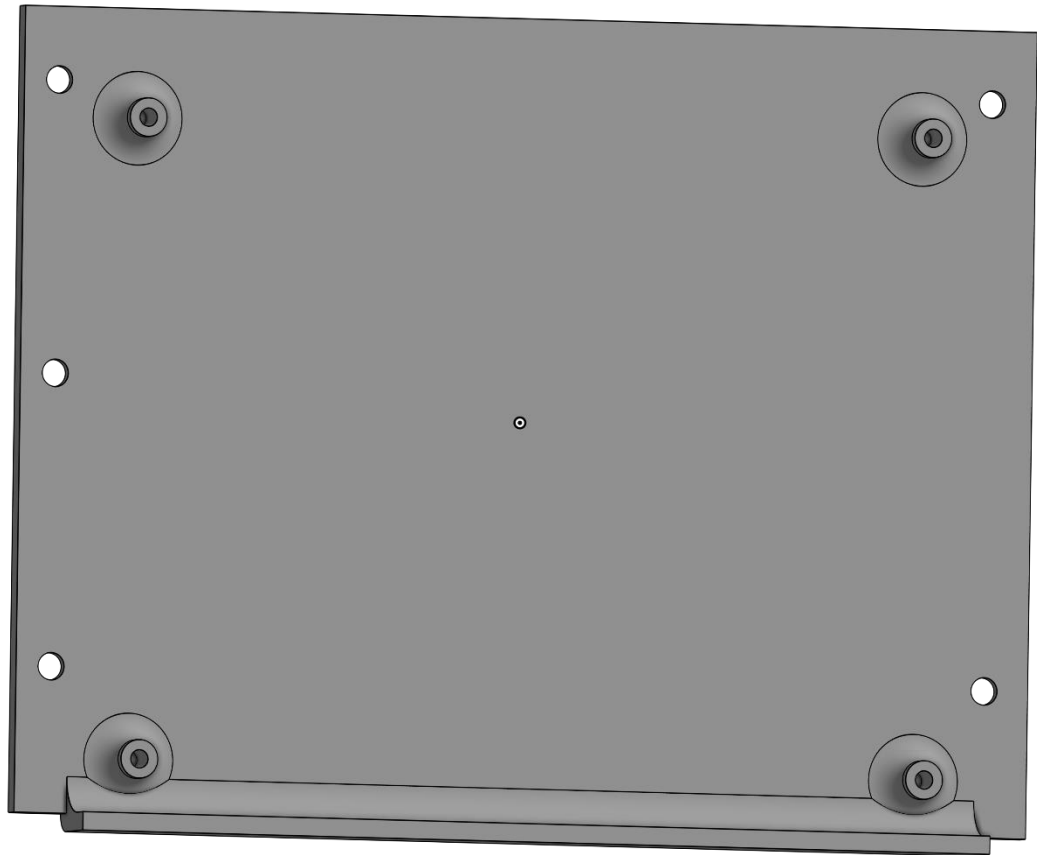
How to upload the firmware

Enclosure

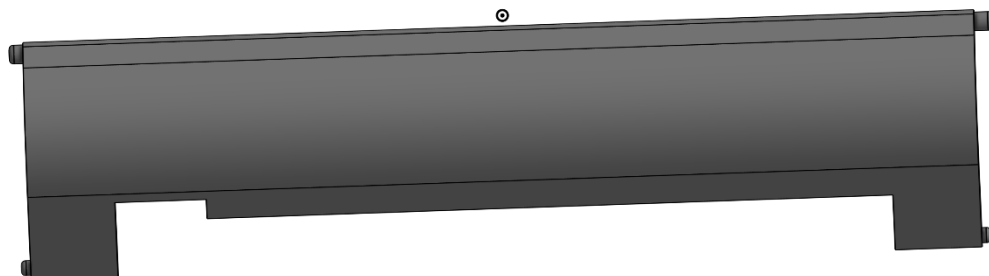
Casing



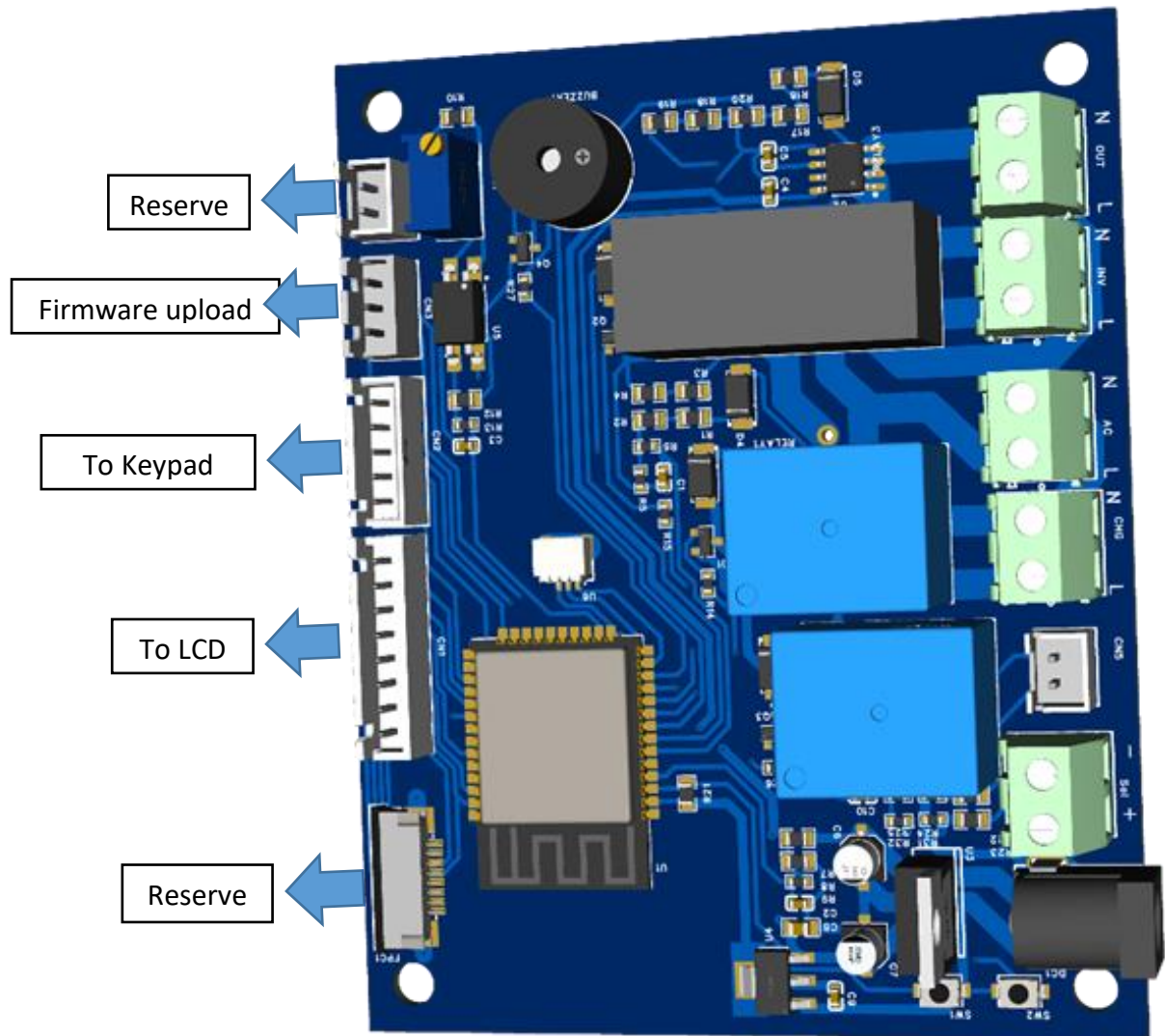
Back plate



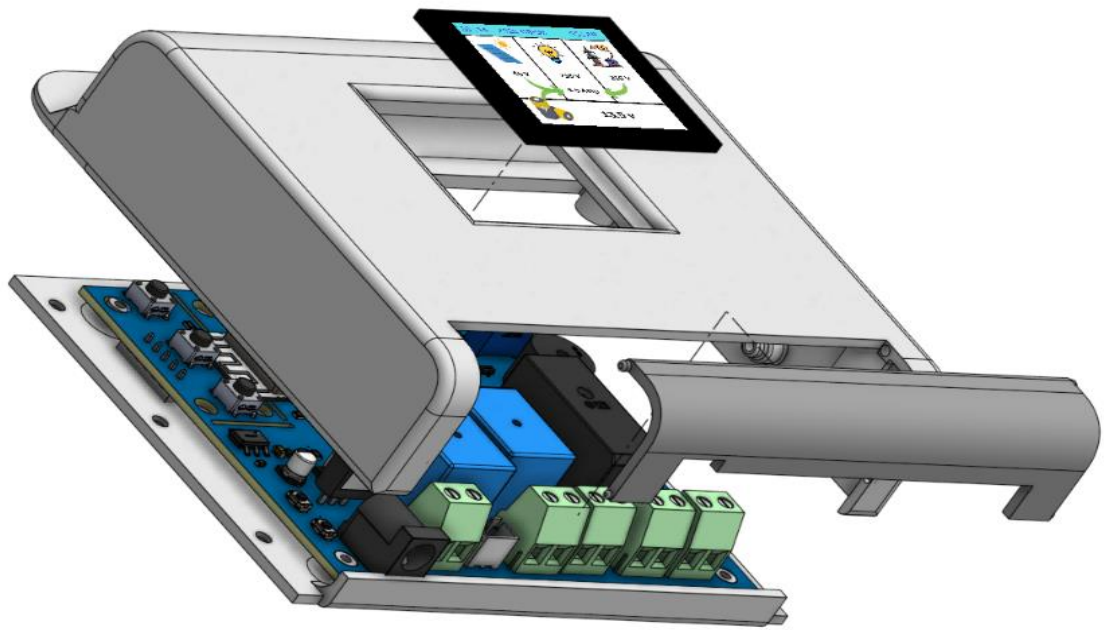
Connection cover



Assembling



Connect 1.8" TFT LCD to **CN1** connector of the mainboard and keypad to the **CN2** connector of the mainboard as mentioned in the figure.



Finished product display

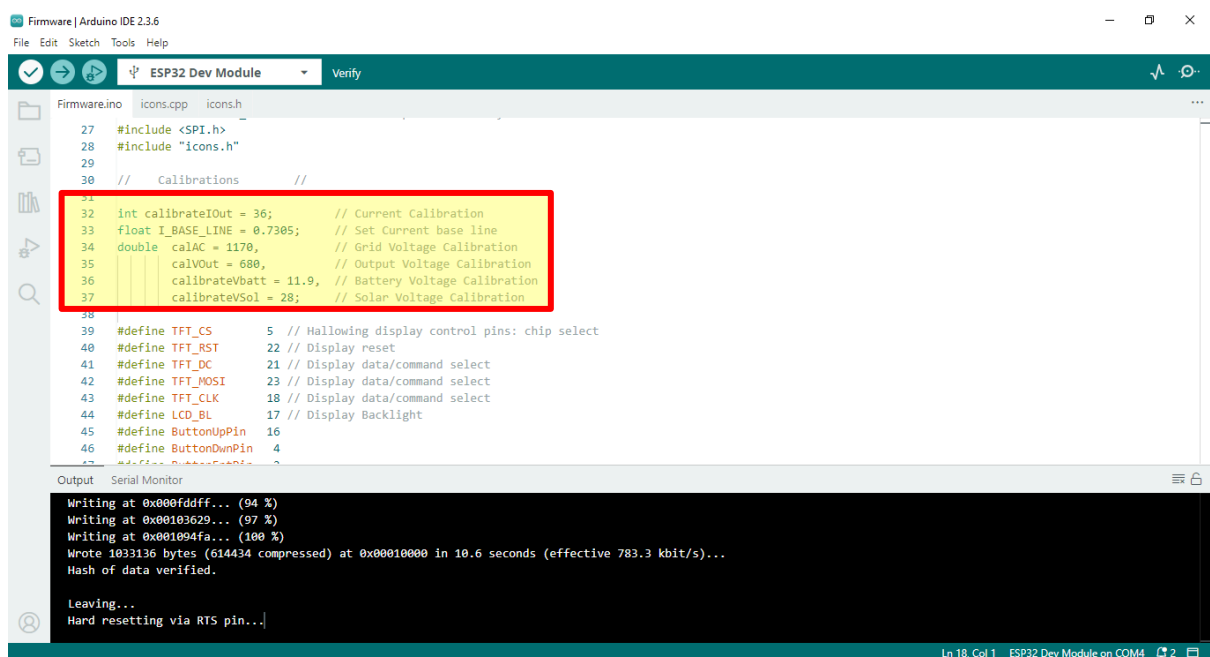


Calibrations

The measurement channels should be calibrated before normal operation.

Calibration may be required for:

- PV voltage
- Battery voltage
- Current sensors
- Grid Voltage
- Other analog measurement channels



```
Firmware | Arduino IDE 2.3.6
File Edit Sketch Tools Help
ESP32 Dev Module Verify

Firmware.ino icons.cpp icons.h
27 #include <SPI.h>
28 #include "icons.h"
29
30 // Calibrations
31
32 int calibrateIOut = 36; // Current Calibration
33 float I_BASE_LINE = 0.7305; // Set Current base line
34 double calAC = 1170, // Grid Voltage Calibration
35 calVOut = 680, // Output Voltage Calibration
36 calibrateVbatt = 11.9, // Battery Voltage Calibration
37 calibrateVSol = 28; // Solar Voltage Calibration
38
39 #define TFT_CS 5 // Hallowing display control pins: chip select
40 #define TFT_RST 22 // Display reset
41 #define TFT_DC 21 // Display data/command select
42 #define TFT_MOSI 23 // Display data/command select
43 #define TFT_CLK 18 // Display data/command select
44 #define LCD_BL 17 // Display Backlight
45 #define ButtonUpPin 16
46 #define ButtonDwnPin 4
47 #define ButtonPin 2

Output Serial Monitor
Writing at 0x00fddfff... (94 %)
Writing at 0x00103629... (97 %)
Writing at 0x001094fa... (100 %)
Wrote 1033136 bytes (614434 compressed) at 0x00010000 in 10.6 seconds (effective 783.3 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...

Ln 18, Col 1 ESP32 Dev Module on COM4 2
```

Use a calibrated multimeter or suitable reference instrument when performing calibration.

The calibration procedure and values should correspond to the actual hardware and firmware version.

Troubleshooting

Problem	Possible Cause
TFT does not start	Wiring, power, SPI configuration, or firmware
Keypad not responding	Wiring or GPIO configuration
Voltage reading incorrect	Calibration or voltage-divider configuration
Current reading incorrect	ACS712 offset/calibration
Relay does not operate	Driver, GPIO, relay supply, or wiring
Grid status incorrect	Grid detection circuit or configuration
ESP32 resets	Power supply, noise, or wiring
Incorrect source switching	Firmware settings or sensing conditions

For detailed troubleshooting, compare the measured hardware signals with the schematic and configuration.

Limitations

This project is designed to work with the hardware configuration documented in the repository.

Performance and measurement accuracy depend on:

- Sensor accuracy
 - Component tolerances
 - PCB design
 - Calibration
 - Power supply quality
 - UPS/inverter characteristics
 - Relay/contact ratings
 - Installation and wiring
-

Future Development

Possible future improvements include:

- Wi-Fi monitoring
- Web dashboard

- Mobile monitoring
 - Energy logging
 - Remote notifications
 - OTA firmware updates
 - Data logging
 - Additional operating modes
 - Improved measurement accuracy
 - Additional battery-management functions
-

Disclaimer

This project is provided “**as is**” for educational, experimental, and DIY purposes.

The system may involve **high battery currents, solar DC voltage, inverter output, and potentially lethal AC mains voltage**. Incorrect wiring, component selection, firmware configuration, or installation can result in electric shock, fire, equipment damage, battery damage, or other hazards.

The author assumes no responsibility for injury, death, equipment damage, fire, property damage, or any other loss resulting from the construction, modification, installation, or use of this project.

Do not work on energized circuits.

If you are not experienced with mains-voltage or high-current electrical systems, seek assistance from a qualified professional.

Project Status

Status: Development / Testing

The hardware and firmware are under continuous development. Features, circuit designs, firmware behavior, and documentation may change between versions.